

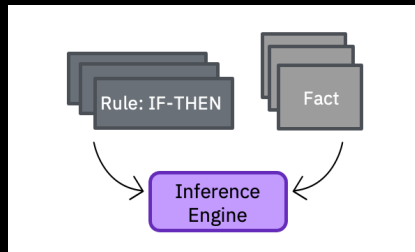
Lecture 7

I-Hsin Chung

Seetharami Seelam

The story of AI is a story of data representations

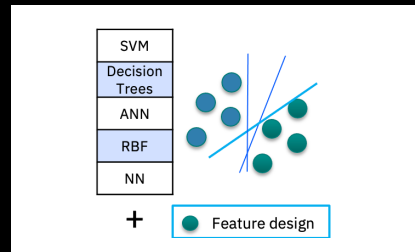
1980s
Big data



Expert Systems

Hand-crafted symbolic representations

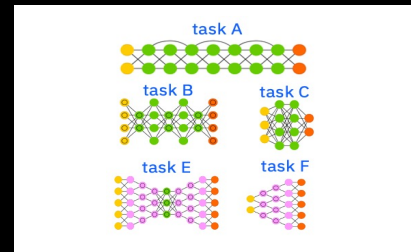
1980s to ~2010
Massive labeled data
+
Compute



Machine Learning

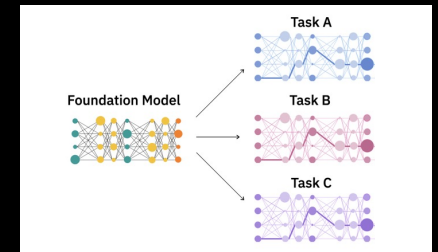
Task-specific hand-crafted feature representations

2017+
Self-supervision at scale
+
Massive unlabeled data
+
Compute



Deep Learning

Task-specific learnt feature representations



Foundation Models

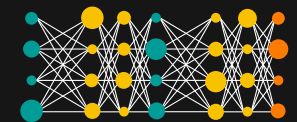
Generalizable & adaptable learnt representations

Foundation models: Generalizable and adaptable

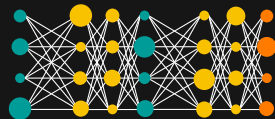
 Self-supervised
training



Foundation model



 Task - specific
model tuning



+



Translation
model



Summarization
model



Answer finding
model

Foundation model advantages



Less effort and lower upfront costs required for data acquisition and labeling

Pre-training dataset reduces incremental data needs and effort (e.g., effort associated with labeling)



Faster deployment time

Skipping initial data gathering, model development, and training steps: effort limited to fine tuning and inferencing (potentially faster than NAS-driven development)



Equal or better accuracy for multiple use cases

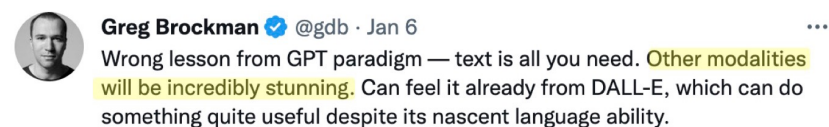
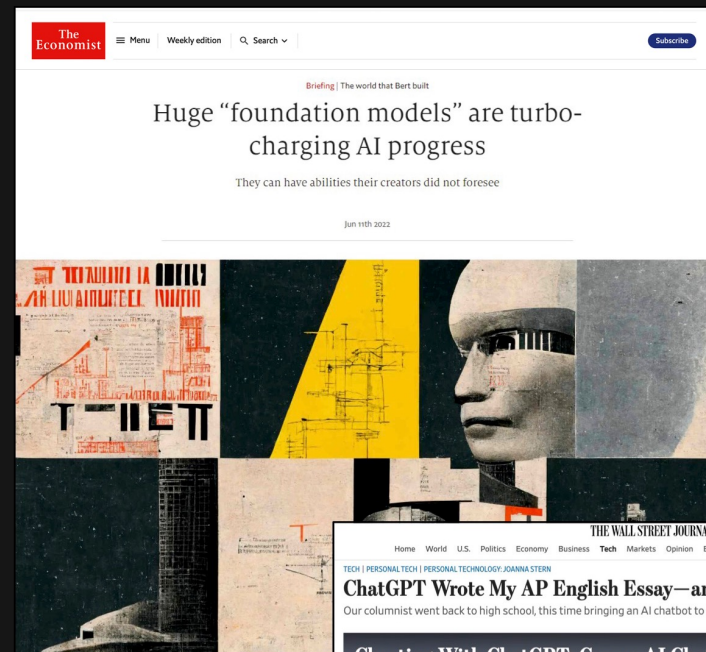
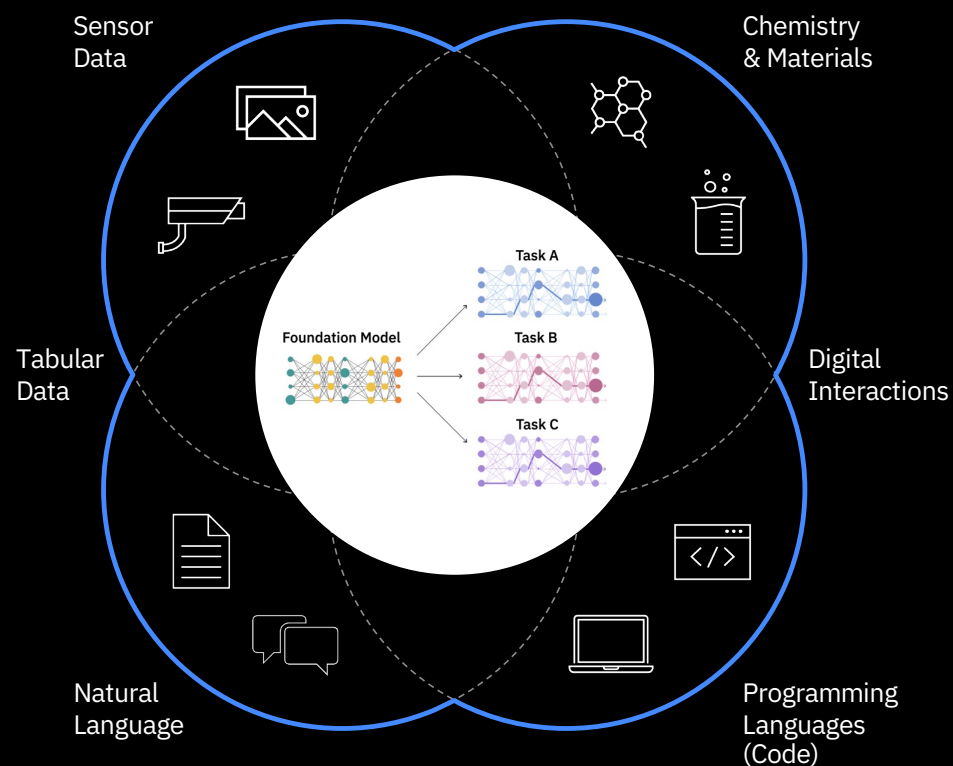
Able to match or improve accuracy vs. state-of-the-art benchmarks across many use cases (potentially better than NAS and other hyperparameter tuning approaches)



Incremental revenue potential

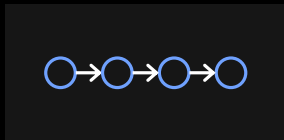
Improved model performance on specific use cases (e.g., molecular discovery) driving incremental revenue

Foundation models are poised to change the game across data modalities and domains



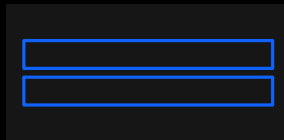
The workflow

Data preparation



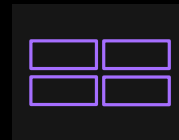
Workflow of steps
(e.g. remove hate
and profanity,
deduplicate, etc.)

Distributed training



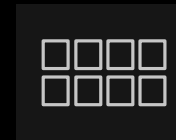
Long-running job on
massive infrastructure

Model adaptation

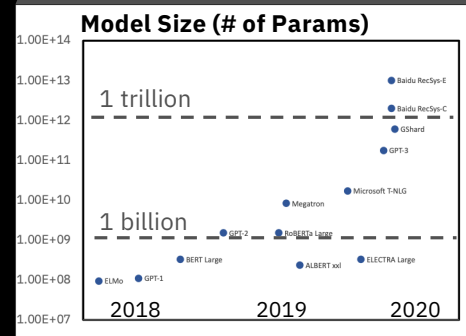


Model tuning with
custom data set for
downstream tasks

Inference



May have sensitivity
to latency,
throughput, power



Foundation Model (FM)

Definition (Stanford CRFM, 2021):

A large machine learning model trained on vast amounts of data (often using self-supervised or semi-supervised learning) that can be adapted to a wide range of downstream tasks.

- **Key Characteristics**

- **Transformer-based architecture**
- Built around the **attention mechanism**
- Composed of **repeated transformer blocks**
- Originated in **NLP**, developed over ~20 years of research progress
- Rapidly increasing model scale → **Large Language Models (LLMs)**

- **Scaling Behavior**

- Within a given model family (e.g., GPT, BERT),
predictive performance improves as model size increases
- Larger models generally exhibit stronger generalization and transfer capability

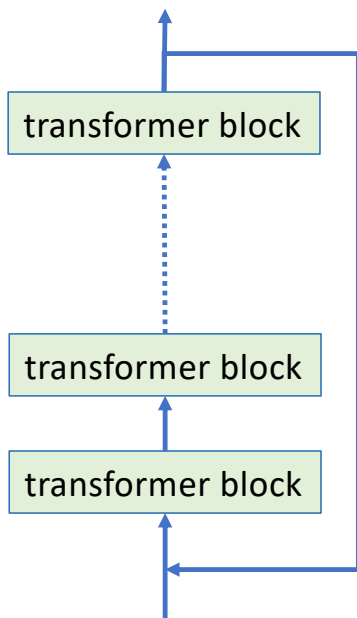
Extensible Usage Pattern of Foundation Models

- **Pretraining + Fine-Tuning (Classic Paradigm)**
 - **Unsupervised pretraining** on massive datasets
 - Followed by **supervised fine-tuning** on task- or domain-specific labeled data
 - **Fine-tuning updates model weights**
- **Pretrain + In-Context Learning (Prompt Engineering)**
 - Use the pretrained model **without changing weights**
 - Guide behavior through:
 - Instructions
 - Context
 - Demonstrations (few-shot / multi-shot examples)
 - **Prompt engineering:**
“Getting the model to do what you want at inference time by providing sufficient context, instructions, and examples — without modifying the underlying weights.”
- **Application Domains**
 - **Traditional NLP Tasks:** Language modeling, Question answering, Reading comprehension, Sentiment analysis, Paraphrasing
 - **Beyond NLP:** Translation, Summarization, Writing assistance, Code generation, Image segmentation, Bio-sequence modeling, ...

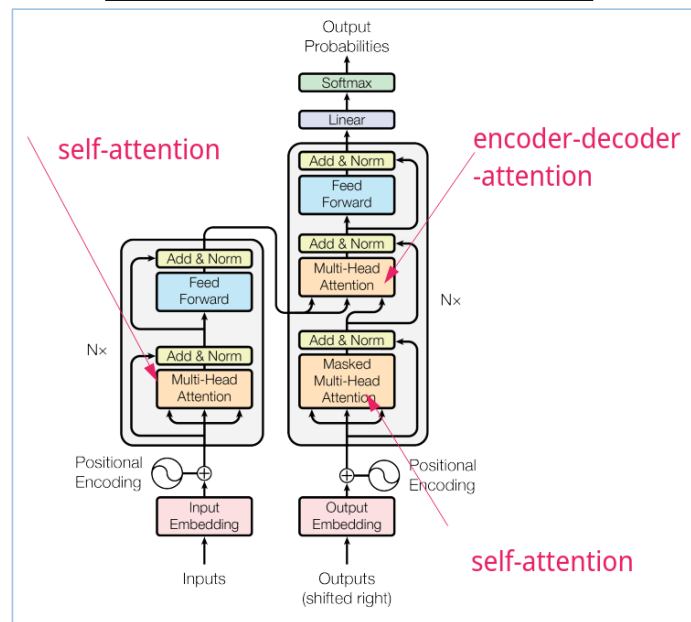
Transformer model architecture

- Repeated transformer blocks, with key configuration parameters:
 - Number of transformer block layers
 - Transformer block: Encoder, decoder, encoder-decoder
 - Embedding (hidden, reduction) dimension size

Typical transformer model



transformer block architecture

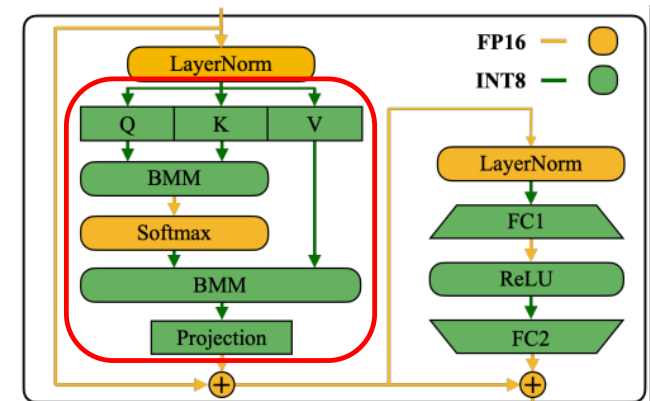


[Attention is all you need \(oversimplified\), Aayush Neupane](#)

Dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

[Attention is.. , Google, June 2017](#)



[Smoothquant, MIT, June 2023](#)

<https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>

Transformer key contributions

- **Self-Attention Mechanism:**
 - Instead of using recurrent (RNN) or convolutional (CNN) structures, the Transformer relies on self-attention to process input sequences in parallel.
 - The **scaled dot-product attention** computes attention weights using queries, keys, and values.
- **Multi-Head Attention:**
 - Multiple attention heads allow the model to focus on different parts of the sequence simultaneously.
- **Positional Encoding:**
 - Since Transformers don't process tokens sequentially like RNNs, they add **positional encodings** to retain the order of words.
- **Encoder-Decoder Structure:**
 - The **encoder** consists of multiple layers where each layer has multi-head self-attention and feed-forward networks.
 - The **decoder** is similar but includes an additional **masked self-attention** mechanism to prevent attending to future tokens during training.
- **Parallelization & Efficiency:**
 - Removing recurrence enables much better parallelization, leading to significant speedup in training compared to RNNs.
- Transformer laid the foundation for later advancements like **BERT, GPT, T5, and other large-scale language models**.

Attention

1. Input embeddings
 - Each word in the input sequence (e.g., a sentence) is first converted into a **word embedding**, which is a high-dimensional vector representing its meaning. Additionally, **positional encodings** are added to retain word order. After embedding, each word is represented as a vector:
 $X=[x_1, x_2, x_3, x_4, x_5, x_6]$ where each x_i is a **d-dimensional embedding**.
2. Compute Queries, Keys, and Values
 - Each attention head applies **three learned weight matrices** to the input embeddings to generate:
 - **Query (Q)**: What this word is looking for. **Key (K)**: What this word contains. **Value (V)**: The actual content to be attended to.
 - Each of these is computed using learned weight matrices W_Q , W_K , and W_V : $Q=XW_Q$, $K=XW_K$, $V=XW_V$
3. Compute Attention Scores
 - by taking the dot product of the **Query** and **Key** matrices: $\text{Scores}=QK^T$
 - The matrix shows how much each word attends to others.
 - Scale the scores (to stabilize gradients): $\text{Scaled Scores} = \frac{QK^T}{\sqrt{d_k}}$ and then apply softmax (row-wise normalization): $\alpha = \text{softmax}(\text{Scaled Scores})$
Meaning of $A(i,j)$ of the Attention matrix - $\dim(\text{seq_len}, \text{seq_len})$
 - The row index i corresponds to the query token (the token currently being processed).
 - The column index j corresponds to the key token (the token being attended to).
 - If $A(i,j)$ is high, token i strongly attends to token j , meaning token j provides a lot of useful context for token i .
 - If $A(i,j)$ is low, token i does not rely much on token j .
4. Compute weighted sum of Values
 - Each word's final representation is obtained by multiplying the **attention weights** by the **Values (V)**: $\alpha \cdot V$
 - This gives an updated vector representation for each word, influenced by the words it attends to. This step allows each word to gather **contextual information** from other words in the sequence.

Attention - continued

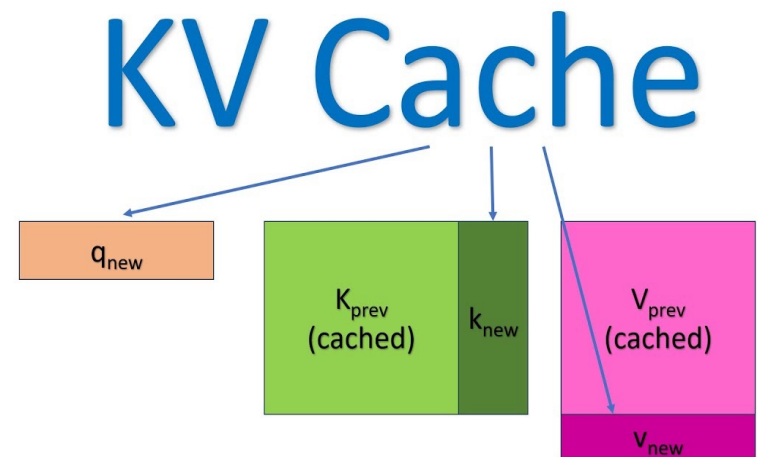
- Self-attention vs. Encoder-decoder attention

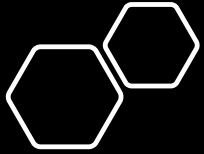
Feature	Self-Attention	Encoder-Decoder Attention
Where it's used	Encoder & Decoder	Only in Decoder
Q, K, V source	All from the same sequence (input or output)	Q from decoder, K & V from encoder
Purpose	Capture relationships within the same sequence	Align decoder output with input

- Target sequence** The correct output sequence that the model is trained to predict. It is used in the **decoder** and the **loss calculation** process.
 - Shift right:** The target sequence is shifted right by one token, and the model learns to predict each token based on the tokens before it. This shift ensures that the model **predicts the next token** while preventing it from using the future tokens during training.
- Input: a sequence of **word embeddings** with **positional encodings**.
 - The sequences are non-overlapping. During **training**, the model typically processes sequences **independently** within each batch, learning dependencies and relationships within each sequence. Information **across sequences** is not directly correlated during training. However, the model is exposed to **a massive corpus** with varied combinations of sequences, allowing it to generalize knowledge.
- Output:
 - For Encoders (BERT, GPT, etc.):**
The output embeddings serve as **contextualized representations** of each token that captures relationships with all other tokens in the sequence, useful for tasks like text classification, named entity recognition, etc.
 - For Decoders (Translation, GPT, etc.):**
The output is a **context-aware representation used to predict the next word, based only on previously generated tokens (and optionally the encoder's output)**, useful for tasks like machine translation.

KV Cache in Transformers (Inference Optimization)

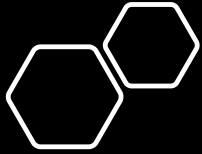
- **Used during autoregressive text generation (inference)**
- Stores previously computed **Keys (K)** and **Values (V)**
- When a new token arrives:
 - Only compute its **Query (Q)**
 - Reuse cached **K and V**
- Avoids recomputing attention over the full sequence
- Reduces per-step computation from $O(n^2)$ to $O(n)$
- Enables:
 - Faster decoding
 - Lower latency
 - Efficient long-context generation
- **Not used during training** (full sequence processed in parallel)





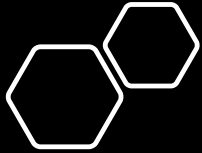
Metrics of performance

- Basic metrics - what can be measured directly
 - The duration of some time interval
 - Time spent in a function
 - Time to transmit a file
 - The count of an event
 - Number of L1 cache misses
 - Number of messages sent
 - The size of some parameter
 - Size of the memory used



Metrics of performance

- Derived metrics - calculation using measured metrics
 - CPI – cycles per instruction
 $\text{time (cycles)} / \# \text{ instructions}$
 - IPC – instructions per cycle
 $\# \text{ instructions} / \text{time (cycle)}$
 - Bandwidth utilization
 $\# \text{ bytes} / \text{time}$
 - FLOPS - floating point operations executed per second
 $\# \text{ float_point_operations} / \text{time}$



Execution time

- Real/Wall clock/Elapsed time:

Actual elapsed time from a point in the past

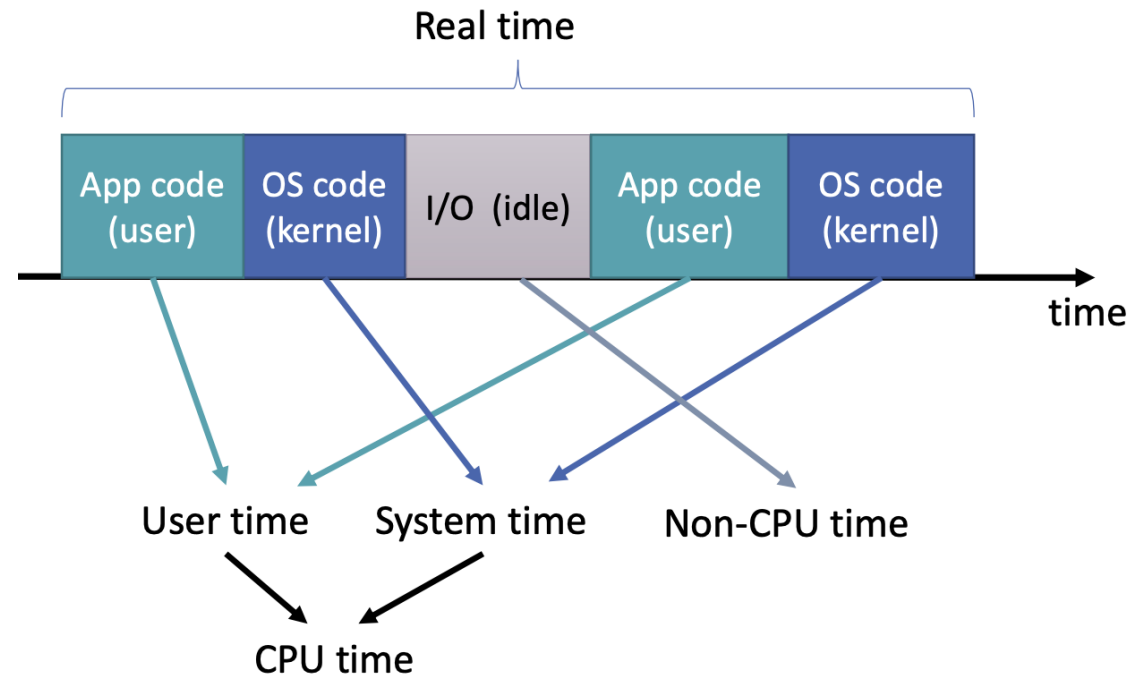
- CPU/Process time:

Time spent executing CPU instructions

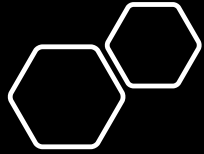
- User time: time spent in user space
- System time: time spent in kernel (OS) space

- Non-CPU time:

Time spent waiting (CPU is idling) for I/O, virtualization, etc.



Morari & Domeniconi - NYU & IBM

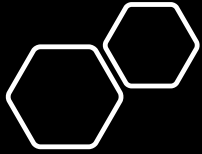


Time Measurement - Linux

- time command - Real, User and System times

```
$ time ./a.out  
real 0m2.450s  
user 0m0.430s  
sys 0m0.000s
```

- millisecond granularity, accuracy may vary between systems!
- $\text{real} \geq \text{user} + \text{sys} + \text{non-CPU time}$



Time measurement - Python

- Real Time:

```
import time
start=time.monotonic()

....
end=time.monotonic()
print("time: " + str(end-start))
```
- granularity fractions of seconds – printing in seconds
- `time.monotonic_ns()` (granularity in nanoseconds)
- <https://docs.python.org/3/library/time.html>

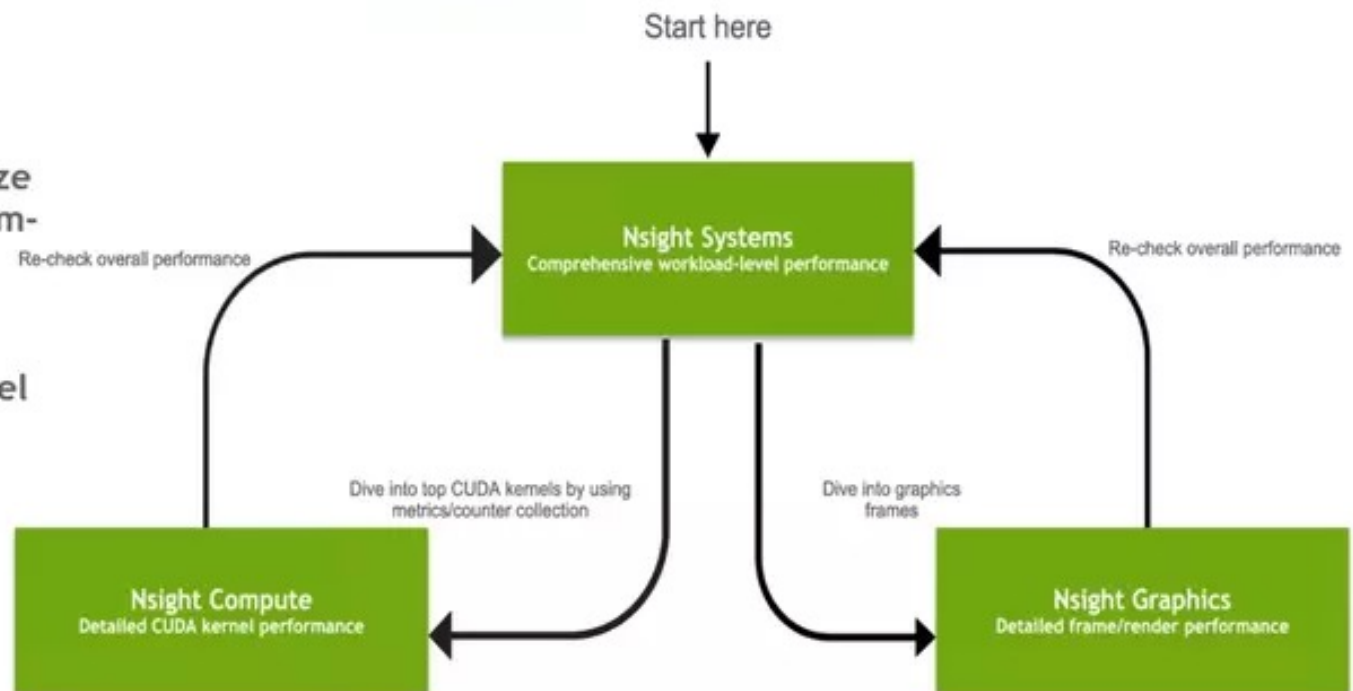
Nsight Product Family

Workflow

Nsight Systems - Analyze application algorithm system-wide

Nsight Compute - Debug/optimize CUDA kernel

Nsight Graphics - Debug/optimize graphics workloads



NVIDIA Nsight Compute

- An interactive kernel profiler for CUDA applications.
- It provides detailed performance metrics and API debugging via a user interface and command line tool.
- Introduction with SDK interface <https://www.nvidia.com/en-us/on-demand/session/other2024-compnsight/?playlistId=playList-c9450de5-2ffd-4ea9-8a1b-24aeeaf49d4e>
- Command line interface (CLI)
 - <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html>
- Performance metrics guide
 - <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html#metrics-guide>
- Roofline charts
 - <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html#roofline-charts>

Nvidia GPU FLOPs measurement

- C:

```
#include <cuda_profiler_api.h>
cudaProfilerStart();
myKernel<<<...>>>(...);
cudaProfilerStop();
```

- Python:

```
import torch.cuda.profiler as profiler
profiler.start()
...
profiler.stop()
```

- Nsight profiler command

```
ncu --profile-from-start off --metrics <comma separated list> --target-processes all <original job command>
```

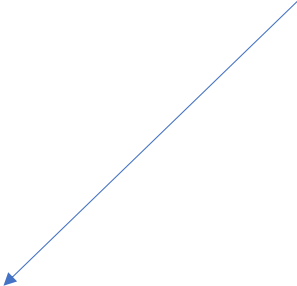
- nvprof command (predecessor of Nsight)

```
nvprof --profile-from-start off --metrics flop_count_sp --profile-child-processes <original job command>
```

- Why different from the estimation?

flop_count_sp:

```
smsp__sass_thread_inst_executed_op_fadd_pred_on.sum +
smsp__sass_thread_inst_executed_op_fmuls_pred_on.sum +
smsp__sass_thread_inst_executed_op_ffma_pred_on.sum * 2
```



Nvidia GPU data transferred metric examples

- NVprof
 - pcie_total_data_received
 - pcie_total_data_transmitted
 - dram_read_bytes/dram_read_throughput
 - dram_write_bytes/dram_write_throughput
- NSight/ncu
 - pcie__read_bytes.sum
 - pcie__write_bytes.sum
 - dram__bytes_read.sum/dram__bytes_read.sum.per_second
 - dram__bytes_write.sum/dram__bytes_write.sum.per_second
- <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html#nvprof-metric-comparison>

Neural network memory complexity

- Memory for parameters
 - Fully connected layers
 - $\text{\#weights} = \text{\#outputs} \times \text{\#inputs}$
 - $\text{\#biases} = \text{\#outputs}$
- Memory for layer outputs
 - $\text{\#outputs}$
- Backward propagation specific
 - Memory for Errors
 - Memory for parameter gradients
 - Memory for hyperparameter-related (e.g., momentum)
- Implementation overhead

What about convolution layers?

What about pooling layers?

What about batch size?

Model Summary in PyTorch

- **pip install torchsummary**

- MNIST

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
from torchsummary import summary
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
        self.conv2_drop = nn.Dropout2d()
        self.fc1 = nn.Linear(320, 50)
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x):
        x = F.relu(F.max_pool2d(self.conv1(x), 2))
        x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
        x = x.view(-1, 320)
        x = F.relu(self.fc1(x))
        x = F.dropout(x, training=self.training)
        x = self.fc2(x)
        return F.log_softmax(x, dim=1)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # PyTorch v0.4.0
model = Net().to(device)
```

```
summary(model, (1, 28, 28))
```

```
-----
Layer (type) Output Shape          Param #
=====
Conv2d-1 [-1, 10, 24, 24]          260
Conv2d-2 [-1, 20, 8, 8]             5,020
Dropout2d-3 [-1, 20, 8, 8]          0
Linear-4 [-1, 50]                   16,050
Linear-5 [-1, 10]                   510
=====
```

```
Total params: 21,840
Trainable params: 21,840
Non-trainable params: 0
```

```
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.06
Params size (MB): 0.08
Estimated Total Size (MB): 0.15
-----
```

Nvidia GPU memory utilization measurement

- nvidia-smi
- gpustat
 - a Python tool that provides detailed stats for your GPU in an easy-to-read format.
 - pip install gpustat
 - gpustat

- Pytorch CUDA API

- cat gpumem.py

```
import torch
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
if device.type == 'cuda':
    print(torch.cuda.get_device_name(0))
    print('Memory Usage:')
    print('Allocated:', round(torch.cuda.memory_allocated(0)/1024**3,1), 'GB')
    print('Reserved: ', round(torch.cuda.memory_reserved(0)/1024**3,1), 'GB')
```

- Python3 gpumem.py

```
Memory Usage:
Allocated: 0.0 GB
Reserved:  0.0 GB
```

Nvidia GPU memory utilization measurement

- GPUUtil python package

- pip3 install gputil psutil humanize
- cat memreport.py

```
# Import packages
```

```
import torch
```

```
import os,sys,humanize,psutil,GPUtil
```

```
import torchvision.models as models
```

```
def mem_report():
```

```
    print("CPU RAM Free: " + humanize.naturalsize( psutil.virtual_memory().available ))
```

```
    GPUs = GPUtil.getGPUs()
```

```
    for i, gpu in enumerate(GPUs):
```

```
        print('GPU {:d} ... Mem Free: {:.0f}MB / {:.0f}MB | Utilization {:.3.0f}%'.format(i, gpu.memoryFree, gpu.memoryTotal, gpu.memoryUtil*100))
```

```
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)
```

```
if torch.cuda.is_available():
```

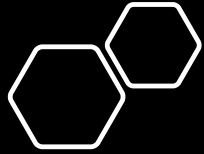
```
    wide_resnet50_2.cuda()
```

```
mem_report()
```

- python3 memreport.py

CPU RAM Free: 244.9 GB

GPU 0 ... Mem Free: 44246MB / 45556MB | Utilization 3%



Performance modeling: Predicting behavior

Identify performance bottleneck

- Determine hardware limit
- Motivate algorithm improvement

Project future hw/sw performance

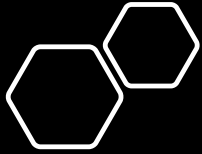
Simplest models: scaling properties

- What parts of the code are serial and parallel?
- How much time is spent in each?
- How efficient are they currently?

More complex concepts

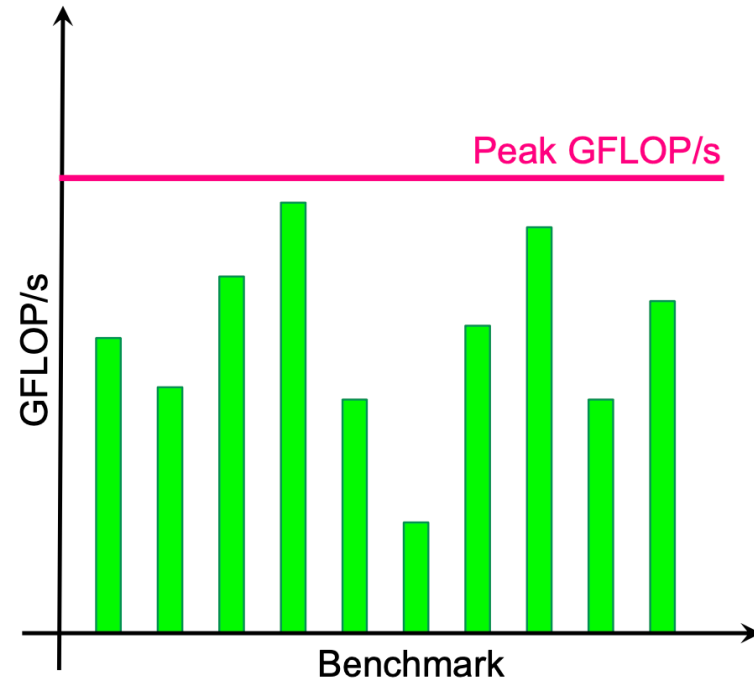
- Roofline model (comparing throughput to theoretical maxima)
- Load balancing: what code is responsible for idle resources?
- Critical path analysis (e.g. Scalasca)

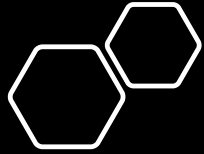
VI-HPS



Are we getting good performance?

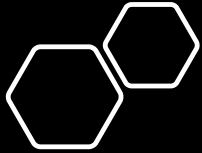
- When profiling GPU-accelerated applications...
- Want to know if the system is well-utilized by the
- GFLOP/s alone may not provide enough insight





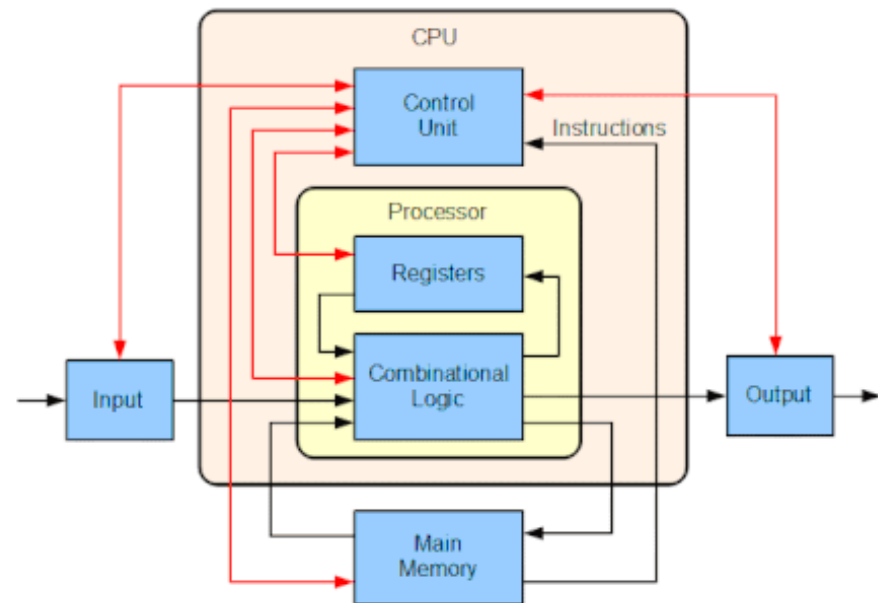
Baseline performance for comparison?

- Using the performance measurements from CPU
- Speedup may not be consistent and may be random
 - GPU isn't always x times faster than a CPU
 - What does Speedup tell us about architecture hardware or application algorithm?
- Speedup provides not enough insights into architecture, application algorithm.
- Speedup provides no guidance system designer nor application users.



Using simulation to understand the performance?

- Modern architectures are incredibly complex
- What to simulate and what not to simulate may be challenging
- Even the simulator can perfectly reproduce the performance
 - may still be hard to interpret and reason the performance factors
 - Huge amount of simulated data, extended time to run simulation worse, (e.g., might incur $10^6\times$ slowdowns)



Data movement versus computation

- Job execution time is the longer of
 - Data movement
 - Computation

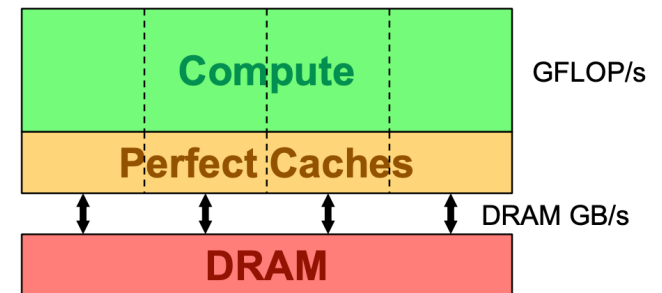
$$\text{Time} = \max \left\{ \begin{array}{l} \frac{\# \text{bytes}}{\text{GB/s}} \\ \frac{\# \text{FP operations}}{\text{GFLOP/s}} \end{array} \right.$$

Data size

System Peak Bandwidth

System Peak Computation capability

Number of calculations



How many FP ops can be completed in a given time?

- $$\text{Time} = \max \left\{ \frac{\frac{\# \text{bytes}}{\text{GB/s}}}{\frac{\# \text{FP operations}}{\text{GFLOP/s}}} \right\} \xRightarrow{\text{Inverse}} \frac{1}{\text{Time}} = \min \left\{ \frac{\frac{\text{GB/s}}{\# \text{bytes}}}{\frac{\text{GFLOP/s}}{\# \text{FP operations}}} \right\}$$

How many calculations per each data byte –
Arithmetic Intensity (AI)
- Multiply by # FP operations*

$$\frac{\# \text{FP operations}}{\text{Time}} = \min \left\{ \frac{\frac{\# \text{FP operations}}{\# \text{bytes}} * \text{GB/s}}{\text{GFLOP/s}} \right\}$$

System Peak Bandwidth

System Peak Computation capability

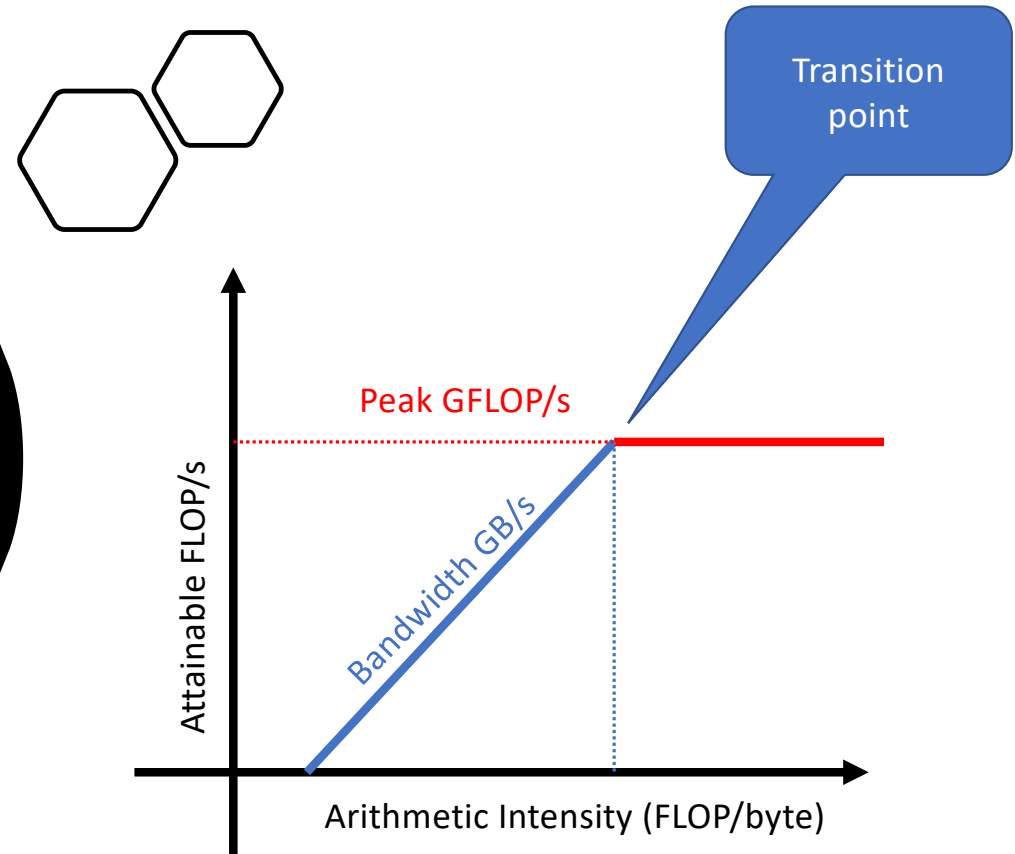
Roofline Model

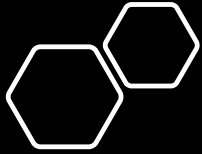
- $\text{Attainable FLOP/s} = \min \begin{cases} AI * \text{peak GB/s} \\ \text{peak GFLOP/s} \end{cases}$
- x axis and y axis are in log scale
- Transition point

$$AI * \text{peak GB/s} = \text{peak GFLOP/s}$$

$$AI = \frac{\text{peak GFLOP/s}}{\text{peak GB/s}}$$

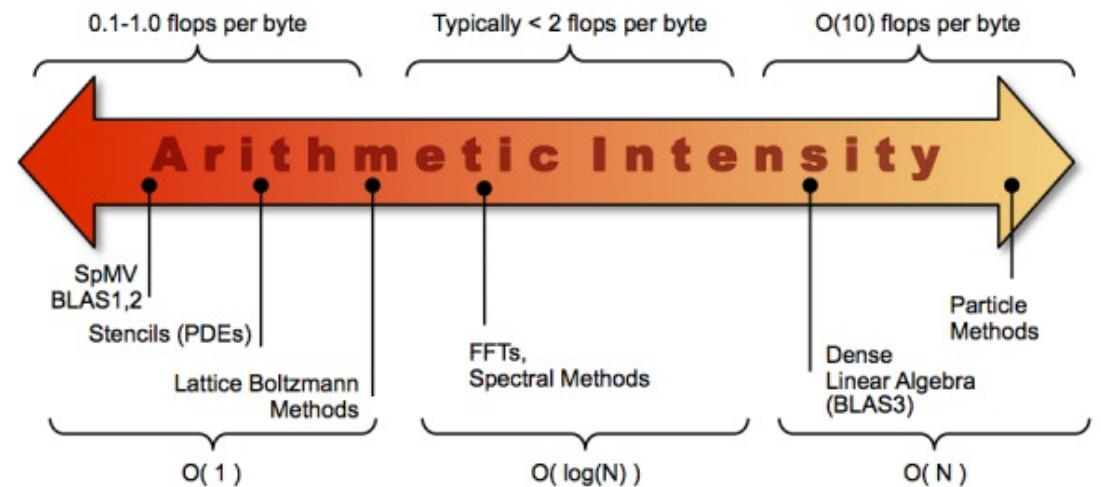
→ Machine is “balanced”

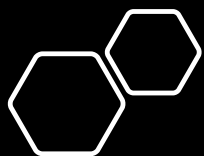




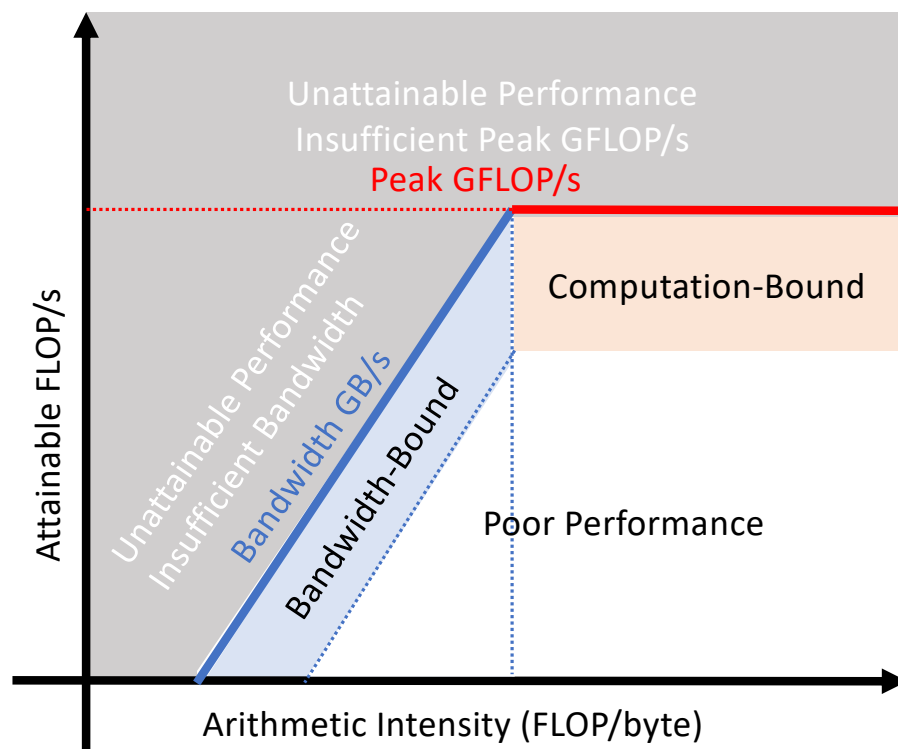
Arithmetic intensity –
compute to comm.
ratio

- The ratio of total floating-point operations (FLOPS) to total data movement (bytes)





Five Regions

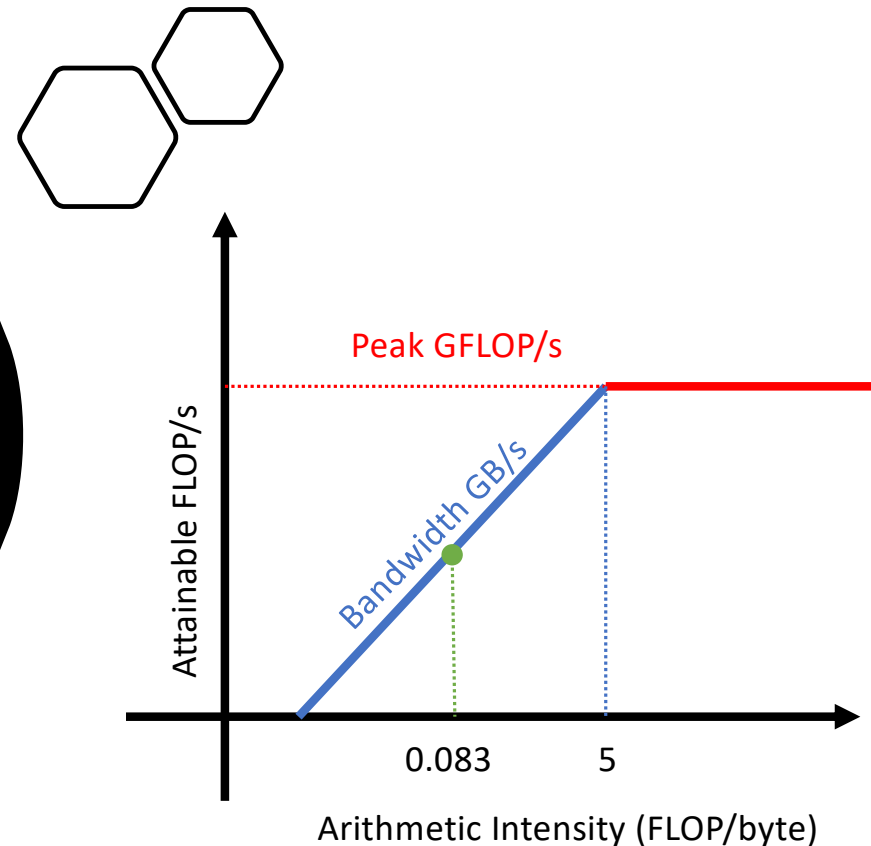


Roofline Example

- Typical machine balance is 5-10 FLOPs/byte
 - 40-80 FLOPs per double to exploit computation capability
 - Artifact of technology and money
 - It is unlikely to improve
- Consider a loop iteration

```
for(i=0;i<n;i++){  
    z[i]=x[i]+alpha*y[i]  
}
```

 - 2 FLOPs per iteration
 - Transfer 24 bytes per iteration (read $x[i], y[i]$, write $z[i]$)
 - AI = 0.083 FLOPs/byte → memory bound

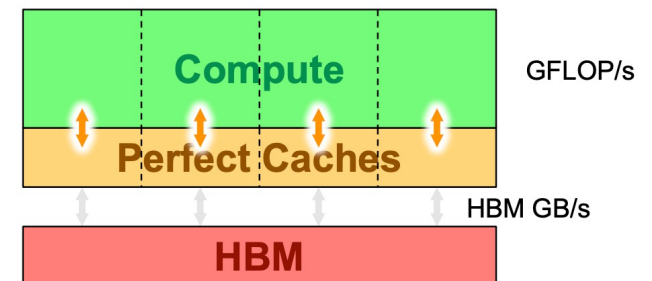


Roofline Example

- 7-point constant coefficient stencil

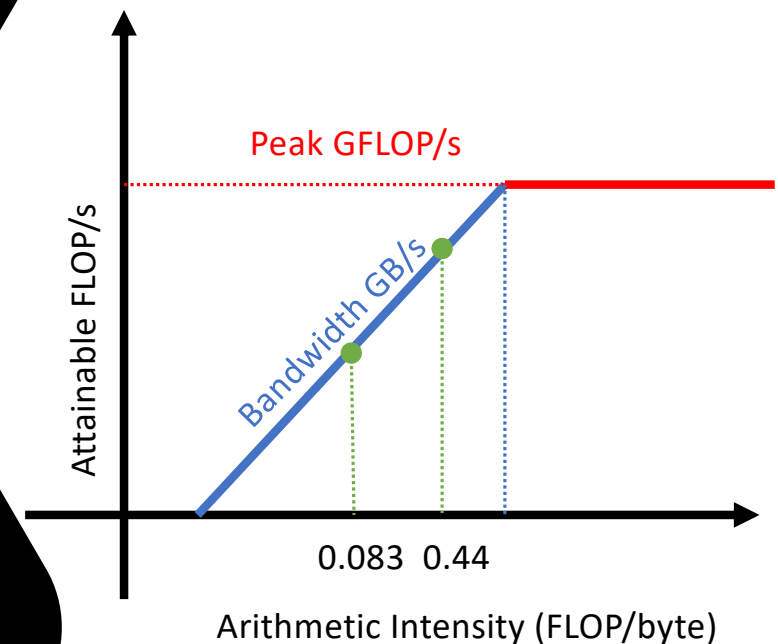
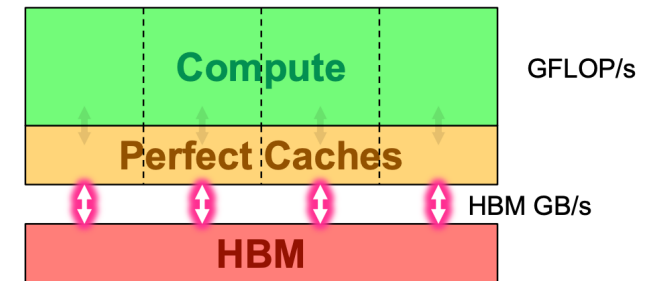
```
for (k=0;k<dim;k++) {  
  for (j=0;j<dim;j++) {  
    for (i=0;i<dim;i++) {  
      new[k][j][i]= -5.0 * old[k ][j ][i ]  
                    + old[k ][j ][i-1] + old[k ][j ][i+1]  
                    + old[k ][j-1][i ] + old[k ][j+1][i ]  
                    + old[k-1][j ][i ] + old[k+1][j ][i ];  
    }  
  }  
}
```

- 7 FLOPS
- 8 memory references (7 reads, 1 write) per point
- $AI = 7 / (8*8) = 0.11$ FLOPs/byte (measured at L1)



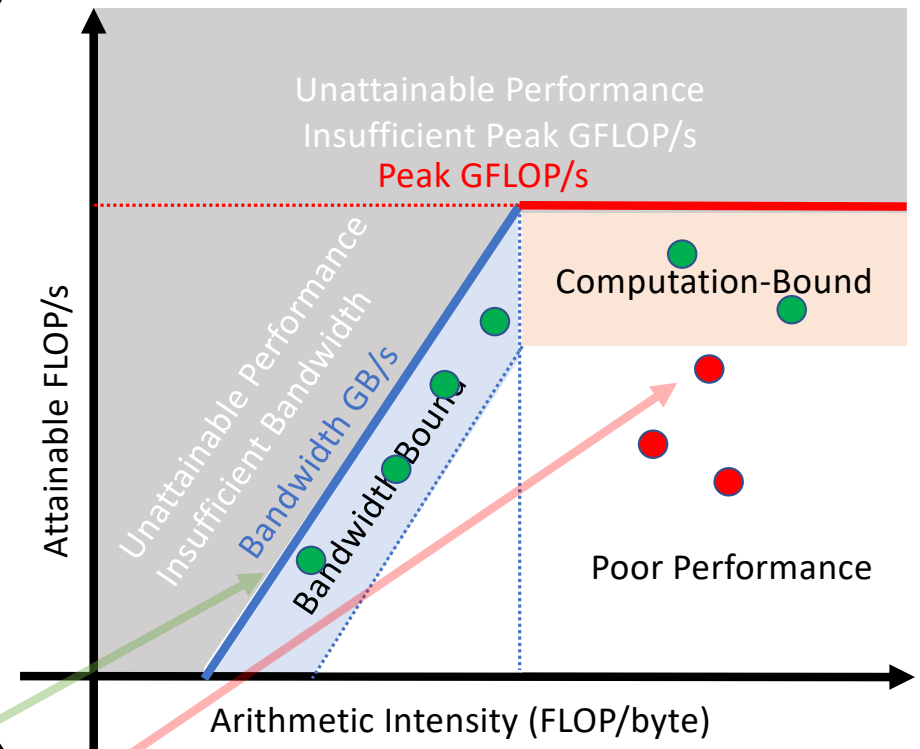
Roofline Example

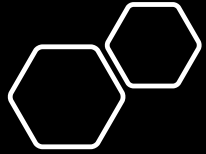
- Cache helps...
- Ideally, cache will filter out all the memory accesses but 1 read and 1 write
- $AI = 7 / (8+8) = 0.44$ Flops/byte
- Still memory bound, but 5x the FLOP rate compared to the previous example



Are we getting good performance?

- Sort benchmarks by AI and plot them into the chart
- Compare the performance against machine capabilities
- Benchmarks close to the rooflines are utilizing the computation resources well
 - Benchmarks can have low performance (GFLOP/s) but make good use of the system
 - Benchmarks can have a high performance (GFLOP/s) but still make poor use of the system





Roofline Recap

Machine Model

- Lines defined by Peak GB/s (bandwidth) and GFlop/s (computation)
- System dependent – unique to each architecture
- Common to all applications running on that system

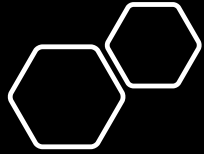
Application Characteristics

- Dots measured by application Gflop's and GB's
- Unique to each application
- Unique to each architecture

Project 1 - Performance modeling and analysis

- Design the experiment to show the difference
 - 2-3 Environment – Cloud vs Baremetal, different CPU/GPU flavors, etc.
 - 2-3 Neural Network models
- Conduct short representative runs; completing the training is unnecessary as accuracy is not the focus.
- Report
 - **Experiment Design (10%)**: Clearly define the objective of the experiment and the hypothesis being tested.
 - **Complexity Estimation (20%)**: Estimate the complexity of the workload
 - **Measurement (15%)**: Collect data to evaluate the performance based on the needed metrics.
 - **Roofline Modeling (20%)**: Create roofline models using the collected measurements (mandatory), with the optional use of Nsight's built-in roofline tool for comparison.
 - **Analysis (35%)**
 - Provide an in-depth analysis of the performance metrics, focusing on how each environment influences the performance.
 - Discuss the influence of NN model choices on the roofline model and performance outcomes
- Due Mar. 27 11:59pm

Different environment		
Different NN models		



Project 1 – ImageNet and Roofline tool

- Pytorch code:
<https://github.com/pytorch/examples/tree/master/imagenet>
- ImageNet 1k data: <http://www.image-net.org/>
 - You don't need the whole data set - just extract some are good enough.
- May use Nsight Roofline
 - <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html#roofline>